

LlamaEngine Architecture (Principal Engineer One-Pager)

Repository: /Users/andreascarlen/GameFusion/Applications/LlamaEngine Date: 2026-02-14

1) What this app is

- LlamaEngine is a cross-platform C++ runtime library wrapping llama.cpp as a stable C ABI.
- EchoLlama is a Qt desktop client that loads that runtime dynamically and provides chat + model download UX.
- The design goal is local-first inference with backend portability (CPU/CUDA/Vulkan/Metal) and optional vision.

2) Layered architecture

- UI layer: EchoLlama (Qt Widgets) handles user input, rendering, model selection, and background tasks.
- Client adapter: LlamaClient resolves symbols from LlamaEngine shared libs via dlopen/LoadLibrary and forwards calls.
- C API boundary: LlamaEngine.cpp exports extern "C" functions for load/generate/session/GGUF parsing.
- Runtime core: LlamaRuntime owns llama_model, session contexts, samplers, prompt formatting, token loop, and callbacks.
- Vision extension: LlamaRuntimeVision loads CLIP/mmpoj and enables image+text generation path.
- Model I/O: NetworkUtils + DownloadManager support resumable Hugging Face downloads in EchoLlama.

3) Runtime flow (text generation)

- A) EchoLlama::processPrompt() runs response work on a QThread (ResponseWorker) to keep UI responsive.
- B) LlamaClient::generateResponse() invokes exported generateResponse(sessionId, ...).
- C) LlamaEngine delegates to singleton runtimeContext (LlamaRuntime).
- D) LlamaRuntime appends user turn, applies llama chat template, tokenizes, decodes, samples token-by-token.
- E) Stream callback emits tokens to UI; final callback returns the complete assistant response.
- F) Session state (messages + KV cache position) persists by integer session ID.

4) Build and packaging model

- Primary project files in this repo: LlamaEngine.pro (shared library), EchoLlama/EchoLlama.pro (GUI app).
- LlamaEngine links llama.cpp libs and backend-specific components selected by BACKEND in qmake.
- EchoLlama expects packaged runtime libraries under Resources/llama.cpp/<llama_version>/<backend>/.
- Workspace convention note: you stated the compile/open entry is build-vs2019-qt6 (external to this repo).
- In this repository snapshot, no local build-vs2019-qt6 folder exists; qmake/QtCreator projects are present.

5) Key engineering observations

- Global runtimeContext in LlamaEngine is simple but limits thread safety and multi-model concurrency.
- Session management is explicit and works, but API docs still mention UUID while implementation uses int IDs.
- generateResponseWithImagePixels currently returns false (stub), so pixel-based multimodal path is incomplete.
- Build scripts and hardcoded paths indicate local environment coupling; CMake + standardized packaging would de-risk CI.